

Conteneurs Docker – Nginx et Apache

Déploiement de serveurs web en conteneurs – IPSSI SQY – BTS SIO SISR

Établissement	IPSSI SQY
Contexte	Projet réalisé en cours – BTS SIO SISR
Outils	Docker, docker-compose, Nginx, Apache (httpd)
OS hôte	Ubuntu Server 24.04 LTS
Durée	1 jour

1. Présentation

Ce projet consiste à déployer des serveurs web Nginx et Apache dans des conteneurs Docker. L'objectif est de comprendre la conteneurisation, la gestion des ports, des volumes et la configuration des serveurs web dans un environnement isolé.

Serveur	Point fort	Usage typique
Nginx	Très performant pour contenus statiques et proxy	Sites web, reverse proxy, load balancer
Apache	Excellent support PHP via mod_php	Applications PHP, WordPress, GLPI...

2. Installation de Docker

```
sudo apt update

sudo apt install ca-certificates curl gnupg -y

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
$VERSION_CODENAME) stable" | sudo tee /etc/apt/sources.list.d/docker.list

sudo apt update && sudo apt install docker-ce docker-ce-cli containerd.io
docker-compose-plugin -y
```

```
sudo usermod -aG docker $USER # pour utiliser Docker sans sudo
```

3. Déploiement d'un conteneur Nginx

```
# Lancer un conteneur Nginx simple

docker run -d --name nginx-test -p 80:80 nginx

# Vérifier que le conteneur tourne

docker ps

# Accéder à la page Nginx : http://localhost dans le navigateur

# Avec un volume pour servir ses propres fichiers HTML

docker run -d --name nginx-custom -p 80:80 -v
/home/lucie/monsite:/usr/share/nginx/html nginx
```

4. Déploiement d'un conteneur Apache

```
# Apache sur le port 8080 (80 déjà pris par Nginx)

docker run -d --name apache-test -p 8080:80 httpd

# Accéder : http://localhost:8080

docker logs apache-test # voir les logs du conteneur
```

5. Fichier docker-compose.yml

Docker Compose permet de gérer plusieurs conteneurs simultanément depuis un seul fichier de configuration.

```
# docker-compose.yml

version: '3.8'

services:

  nginx:
```

```
image: nginx:latest

container_name: nginx

ports:

- '80:80'

volumes:

- ./nginx/html:/usr/share/nginx/html

- ./nginx/conf:/etc/nginx/conf.d

restart: unless-stopped


apache:

image: httpd:latest

container_name: apache

ports:

- '8080:80'

volumes:

- ./apache/html:/usr/local/apache2/htdocs

restart: unless-stopped
```

```
docker compose up -d # démarrer les conteneurs en arrière-plan
```

```
docker compose ps # vérifier l'état
```

```
docker compose down # arrêter et supprimer les conteneurs
```

```
docker compose logs -f # suivre les logs en temps réel
```

6. Configuration d'un Virtual Host Nginx personnalisé

```
# ./nginx/conf/monsite.conf

server {
```

```
listen 80;

server_name monsite.local;

root /usr/share/nginx/html;

index index.html;


location / {

    try_files $uri $uri/ =404;

}


error_log /var/log/nginx/error.log;

access_log /var/log/nginx/access.log;

}
```

7. Commandes Docker utiles

```
docker ps # conteneurs en cours d'exécution

docker ps -a # tous les conteneurs

docker images # images téléchargées

docker exec -it nginx bash # ouvrir un shell dans le conteneur

docker stop nginx && docker rm nginx # arrêter et supprimer

docker system prune # nettoyer les ressources inutilisées
```

8. Points importants à retenir

- **Ports** : deux conteneurs ne peuvent pas écouter sur le même port hôte (Nginx sur 80, Apache sur 8080).
- **Volumes** : permettent de persister les données et de modifier les fichiers sans reconstruire l'image.
- **restart: unless-stopped** : redémarre automatiquement le conteneur si le serveur redémarre.
- **Docker Compose** : indispensable dès qu'on gère plus d'un conteneur.

- **Nginx vs Apache** : Nginx est préféré pour les sites statiques et le reverse proxy, Apache pour PHP.